

SISTEMAS
DISTRIBUIDOS

Practica 8. Microservicios

MICHELLE ANGUIANO RODEA

BOLETA: 2022630181

ESCOM

PROFESOR CHADWICK CARRETO
ARELLANO

GRUPO: 7CM1



Instituto
politécnico
nacional

ESCOM

1) Introducción

Esta práctica implementa una solución cliente–servidor basada en el paradigma de microservicios para componer información de dos dominios independientes: Usuarios y Clima. Cada dominio se materializa como un servicio autónomo —con su propio proceso, puerto, estado y contrato HTTP/JSON— que expone un conjunto mínimo de endpoints REST. El cliente web (HTML/CSS/JavaScript) actúa como orquestador, invocando cada servicio y componiendo un resultado integrado para el usuario final.

A diferencia de un enfoque monolítico, la separación en microservicios promueve el desacoplamiento, la escalabilidad independiente y la resiliencia. Si un servicio falla o requiere mantenimiento, los demás pueden continuar operando siempre que se respeten los contratos publicados. Para efectos didácticos y alineación con prácticas previas, se emplea Java 17 con el servidor HTTP ligero del JDK (`com.sun.net.httpserver`) y persistencia en archivos JSON.

Los objetivos formativos incluyen: (i) comprender los principios de diseño detrás de microservicios, (ii) implementar servicios independientes con contratos REST, (iii) realizar orquestación en el cliente, y (iv) evaluar no solo la funcionalidad sino también atributos de calidad como resiliencia, mantenibilidad y facilidad de despliegue.

2) Problemática

- Rigidez del monolito: cambios locales implican reconstruir y desplegar toda la aplicación.
- Escalabilidad no selectiva: picos de demanda en un módulo obligan a escalar componentes innecesarios.
- Disponibilidad acoplada: fallas en un subsistema pueden arrastrar al resto del sistema.
- Evolución asíncrona: distintos ritmos de cambio por equipo o dominio requieren independencia tecnológica.
- Trazabilidad limitada: es más difícil aislar problemas y medir desempeño por dominio en arquitecturas monolíticas.

Se requiere, por tanto, una arquitectura que posibilite despliegue independiente, tolerancia a fallos y contratos de servicio estables que permitan a los clientes componer funcionalidades de múltiples fuentes.

3) Solución

Se diseñaron dos microservicios independientes —UserService y WeatherService— y un cliente orquestador en HTML/JS:

- UserService (puerto 9001):
 - GET /usuarios — lista de usuarios.

- • GET /usuarios/{id} — detalle por identificador.
- • Persistencia: data/users.json.
- WeatherService (puerto 9002):
 - • GET /clima — listado (opcional, diagnóstico).
 - • GET /clima/{ciudad} — clima por ciudad.
 - • Persistencia: data/weather.json.
- Cliente orquestador (HTML/CSS/JS):
 - • Invoca /usuarios → llena selector.
 - • Invoca /usuarios/{id} → obtiene ciudad.
 - • Invoca /clima/{ciudad} → compone la vista con clima.
 - • Manejo de errores: si un servicio no responde, se informa sin romper la UI.

Esta solución distribuye responsabilidades por dominio, permite escalar servicios de forma selectiva y reduce el acoplamiento. La orquestación en el borde (cliente) evita dependencias directas entre servicios.

4) Implementación

4.1 Tecnologías

- Backend: Java 17, módulo `jdk.httpserver` (`HttpServer`, `HttpHandler`, `HttpExchange`).
- Frontend: HTML, CSS, JavaScript (Fetch API).
- Comunicación: HTTP/JSON con métodos GET; CORS habilitado para entorno de laboratorio.
- Persistencia: archivos JSON en disco para simplificar el laboratorio (sin BD externa).

4.2 Estructura de carpetas

```
practica-8/
├── user-service/
│   ├── src/UserService.java
│   └── data/users.json
├── weather-service/
│   ├── src/WeatherService.java
│   └── data/weather.json
└── client/
    ├── index.html
    ├── style.css
    └── app.js
```

4.3 Contratos de API

Servicio	Método	Ruta	Descripción	Ejemplo
UserService	GET	/usuarios	Lista de usuarios	http://127.0.0.1:9001/usuarios
UserService	GET	/usuarios/{id}	Detalle por id	http://127.0.0.1:9001/usuarios/1
WeatherService	GET	/clima	(Opc.) listado de climas	http://127.0.0.1:9002/clima
WeatherService	GET	/clima/{ciudad}	Clima por ciudad	http://127.0.0.1:9002/clima/CDMX

4.4 Lógica de orquestación (cliente)

1. GET /usuarios → llena el <select>.
2. GET /usuarios/{id} → obtiene la ciudad del usuario.
3. GET /clima/{ciudad} → obtiene temperatura y condición.
4. Render de tarjeta combinando usuario + clima.
5. Manejo de errores: mensajes claros si un servicio falla.

4.5 Pasos de ejecución (Windows)

- UserService: cd user-service → javac -encoding UTF-8 -d out src\UserService.java → java -cp ".;out" UserService
- WeatherService: cd weather-service → javac -encoding UTF-8 -d out src\WeatherService.java → java -cp ".;out" WeatherService
- Cliente: abrir client/index.html o servir con python -m http.server 8080 y abrir http://127.0.0.1:8080/

4.6 Seguridad y robustez (básicas)

- CORS habilitado para pruebas; en producción, limitar por origen/métodos.
- Respuestas 404/405 coherentes para rutas/métodos inválidos.
- Aislamiento de estado entre servicios; comunicación solo vía HTTP.
- Registro básico en consola, escalable a observabilidad con métricas y trazas.

4.7 Riesgos y mitigaciones

- Desalineación de contratos: versionar (p. ej., /v1) y documentar cambios.
- Fallas parciales: timeouts y reintentos con backoff; mensajes de error claros en el cliente.

- Crecimiento de tráfico: escalar servicios de forma independiente; considerar un API Gateway.
- Persistencia en archivos: migrar a BD/servicios gestionados para producción.

5) Resultados

Los dos microservicios respondieron correctamente en sus puertos (9001 y 9002), entregando datos JSON. El cliente orquestador compuso la vista combinando usuarios y clima mediante invocaciones secuenciales. Se comprobó la resiliencia al detener WeatherService: UserService permaneció activo y el cliente notificó el fallo sin comprometer el resto de la interfaz.

La solución es extensible: permite añadir escritura (POST/PUT/DELETE), autenticación, paginación y un API Gateway para políticas comunes (CORS, rate limiting, logging).

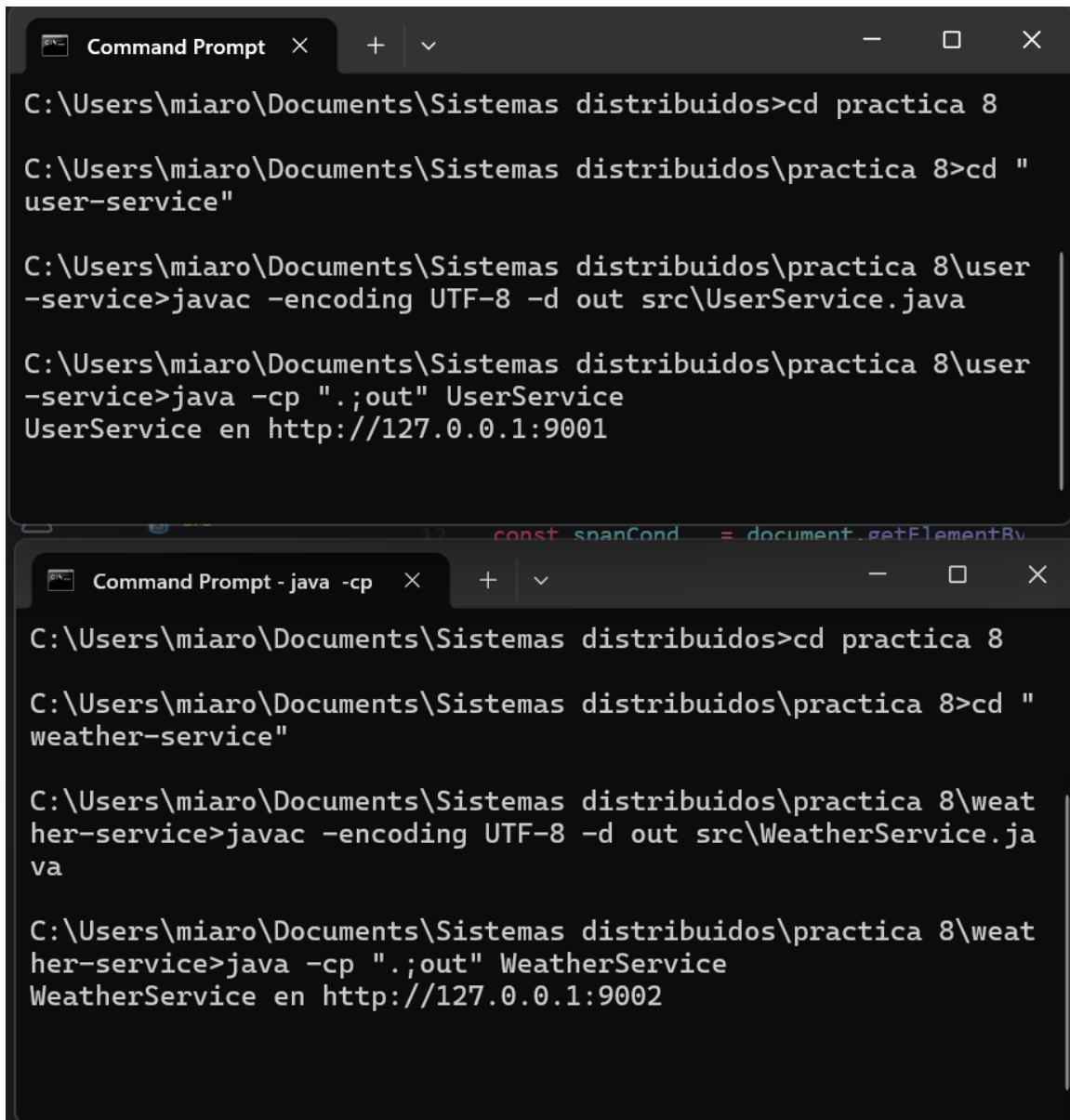
6) Conclusión

La arquitectura implementada demuestra los beneficios de separar dominios en microservicios autónomos con contratos REST. La orquestación en el cliente viabiliza la composición sin acoplar servicios entre sí, favoreciendo la escalabilidad selectiva y la resiliencia. Este trabajo sienta bases para incorporar API Gateway, observabilidad, tolerancia a fallos avanzada y despliegue en contenedores en iteraciones futuras.

7) Referencias

6. MDN Web Docs. "Fetch API". https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
7. MDN Web Docs. "CORS". <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
8. MDN Web Docs. "Service Workers: an overview". https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API
9. W3C. "Web App Manifest". <https://www.w3.org/TR/appmanifest/>
10. Oracle. "com.sun.net.httpserver (JDK httpserver module)". <https://docs.oracle.com/en/java/javase/21/docs/api/jdk.httpserver/com/sun/net/httpserver/package-summary.html>
11. Fowler, M. "Microservices". <https://martinfowler.com/articles/microservices.html>
12. NGINX. "Microservices Reference Architecture". <https://www.nginx.com/learn/microservices/>
13. Google Cloud. "Microservices architecture on Google Cloud". <https://cloud.google.com/architecture/microservices>

Anexo A – Evidencias



The image displays two screenshots of a Windows Command Prompt window, showing the process of compiling and running Java services.

Top Screenshot:

```
Command Prompt
```

```
C:\Users\miaro\Documents\Sistemas distribuidos>cd practica 8

C:\Users\miaro\Documents\Sistemas distribuidos\practica 8>cd "
user-service"

C:\Users\miaro\Documents\Sistemas distribuidos\practica 8\user
-service>javac -encoding UTF-8 -d out src\UserService.java

C:\Users\miaro\Documents\Sistemas distribuidos\practica 8\user
-service>java -cp ".;out" UserService
UserService en http://127.0.0.1:9001
```

Bottom Screenshot:

```
Command Prompt - java -cp
```

```
C:\Users\miaro\Documents\Sistemas distribuidos>cd practica 8

C:\Users\miaro\Documents\Sistemas distribuidos\practica 8>cd "
weather-service"

C:\Users\miaro\Documents\Sistemas distribuidos\practica 8\weat
her-service>javac -encoding UTF-8 -d out src\WeatherService.ja
va

C:\Users\miaro\Documents\Sistemas distribuidos\practica 8\weat
her-service>java -cp ".;out" WeatherService
WeatherService en http://127.0.0.1:9002
```

Microservicios: Usuarios + Clima

Cliente orquestador (fetch a dos servicios).

1. Listar Usuarios

2. Selecciona un usuario ▾

Orquestación en el cliente: compone datos de dos APIs independientes.

Microservicios: Usuarios + Clima

Cliente orquestador (fetch a dos servicios).

1. Listar Usuarios

#1 — Michelle (CDMX) ▾

Hola, Michelle

Reporte del clima para tu ciudad (**CDMX**):

- Temperatura: **22°C**
- Condición: **Soleado**

Orquestación en el cliente: compone datos de dos APIs independientes.

Microservicios: Usuarios + Clima

Cliente orquestador (fetch a dos servicios).

1. Listar Usuarios

#2 — Rubén (Guadalajara) ▾

Hola, Rubén

Reporte del clima para tu ciudad (**Guadalajara**):

- Temperatura: **24°C**
- Condición: **Parcialmente nublado**

Orquestación en el cliente: compone datos de dos APIs independientes.

Microservicios: Usuarios + Clima

Cliente orquestador (fetch a dos servicios).

1. Listar Usuarios

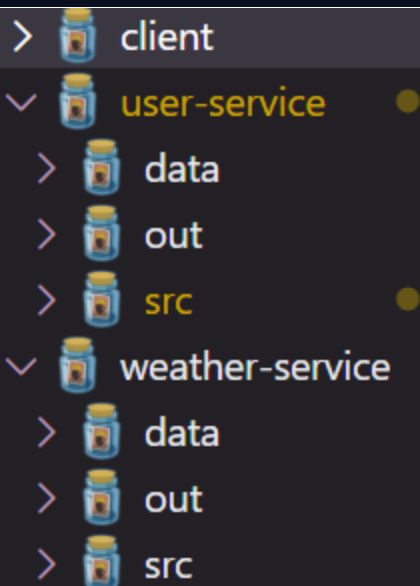
#3 — Sunem (Monterrey) ▾

Hola, Sunem

Reporte del clima para tu ciudad (**Monterrey**):

- Temperatura: **26°C**
- Condición: **Caluroso**

Orquestación en el cliente: compone datos de dos APIs independientes.



```

client > ✨ app.js > ...
 2  const URL_USUARIOS = 'http://127.0.0.1:9001/usuarios';
 3  const URL_CLIMA    = 'http://127.0.0.1:9002/clima/';
 4
 5  const btnListar = document.getElementById('btnListar');
 6  const selectUsuario = document.getElementById('selectUsuario');
 7  const card = document.getElementById('resultsCard');
 8
 9  const spanNombre = document.getElementById('nombre_usuario');
10  const spanCiudad = document.getElementById('ciudad');
11  const spanTemp   = document.getElementById('temperatura');
12  const spanCond   = document.getElementById('condicion');
13  const errorBox   = document.getElementById('error');
14
15  btnListar.addEventListener('click', async () => {
16    try {
17      const res = await fetch(URL_USUARIOS, { cache: 'no-store' });
18      if (!res.ok) throw new Error('Error listando usuarios');
19      const usuarios = await res.json(); // { "1": {...}, "2": {...} }
20      selectUsuario.innerHTML = '<option value="">2. Selecciona un usuario</option>';
21      Object.keys(usuarios).forEach(id => {
22        const u = usuarios[id];
23        const opt = document.createElement('option');
24        opt.value = id;
25        opt.textContent = `#${id} - ${u.nombre} (${u.ciudad})`;
26        selectUsuario.appendChild(opt);
27      });
28      selectUsuario.disabled = false;
29    } catch (err) {
30      alert(err.message);
31    }
  });

```

```
client > * app.js > ...
15 btnListar.addEventListener('click', async () => {
30     alert(err.message);
31 }
32 });
33
34 selectUsuario.addEventListener('change', async (ev) => {
35     const id = ev.target.value;
36     if (!id) { card.style.display = 'none'; return; }
37     errorBox.textContent = '';
38     try {
39         // 1) detalle del usuario
40         const ru = await fetch(`${URL_USUARIOS}/${id}`);
41         if (!ru.ok) throw new Error('Usuario no encontrado');
42         const user = await ru.json();
43
44         spanNombre.textContent = user.nombre;
45         spanCiudad.textContent = user.ciudad;
46
47         // 2) clima por ciudad del usuario
48         const rc = await fetch(`${URL_CLIMA}${encodeURIComponent(user.ciudad)}`);
49         if (!rc.ok) throw new Error('Clima no disponible');
50         const clima = await rc.json();
51
52         spanTemp.textContent = clima.temperatura;
53         spanCond.textContent = clima.condicion;
54
55         card.style.display = 'block';
56     } catch (e) {
57         errorBox.textContent = 'Error: ' + e.message;
55         card.style.display = 'block';
56     } catch (e) {
57         errorBox.textContent = 'Error: ' + e.message;
58         card.style.display = 'block';
59     }
60 });
```

```

1 <!doctype html>
2 <html lang="es">
3 <head>
4   <meta charset="utf-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1" />
6   <title>P8 • Microservicios (Usuarios + Clima)</title>
7   <link rel="stylesheet" href="./style.css" />
8 </head>
9 <body>
10 <header class="wrap">
11   <h1>Microservicios: Usuarios + Clima</h1>
12   <p class="muted">Cliente orquestador (fetch a dos servicios).</p>
13 </header>
14
15 <main class="wrap">
16   <section class="card">
17     <div class="controls">
18       <button id="btnListar">1. Listar Usuarios</button>
19       <select id="selectUsuario" disabled>
20         <option value="">2. Selecciona un usuario</option>
21       </select>
22     </div>
23   </section>
24
25   <section id="resultsCard" class="card" style="display:none">
26     <h2>Hola, <span id="nombre_usuario">...</span></h2>
27     <p>Reporte del clima para tu ciudad (<strong id="ciudad">...</strong>):</p>
28     <ul>
29       <li>Temperatura: <strong id="temperatura">...</strong></li>
30       <li>Condición: <strong id="condicion">...</strong></li>
31     </ul>
32     <div id="error" class="error"></div>
33   </section>
34 </main>
35
36 <footer class="wrap foot">
37   <small>Orquestación en el cliente: compone datos de dos APIs independientes.</small>
38 </footer>
39
40 <script src="./app.js"></script>
41 </body>
42 </html>

```

client > style.css > ...

```

1 :root { --bg: #0b1020; --card: #111634; --text: #e8ecff; --muted: #9aa3c7; --accent: #0ea5e9; }
2 * { box-sizing: border-box; }
3 html, body { margin: 0; background: var(--bg); color: var(--text); font-family: system-ui, Segoe UI, Roboto, sans-serif; }
4 .wrap { max-width: 900px; margin: 16px auto; padding: 0 16px; }
5 .muted { color: var(--muted); }
6 .card { background: var(--card); border: 1px solid #1f2758; border-radius: 16px; padding: 16px; margin-bottom: 16px; box-shadow: 0 4px 12px #000000; }
7 .controls { display: flex; gap: 8px; }
8 button { padding: 10px 14px; background: var(--accent); color: #06101f; border: none; border-radius: 12px; font-weight: 600; }
9 select { padding: 10px 14px; border-radius: 12px; border: 1px solid #23307b; background: #0e1440; color: var(--text); }
10 .foot { text-align: center; color: var(--muted); }
11 .error { margin-top: 8px; color: #ffb4b4; }
12

```

user-service > data > users.json > ...

```
1 {
2   "1": { "id": "1", "nombre": "Michelle", "ciudad": "CDMX" },
3   "2": { "id": "2", "nombre": "Rubén", "ciudad": "Guadalajara" },
4   "3": { "id": "3", "nombre": "Sunem", "ciudad": "Monterrey" }
5 }
```

user-service > src > UserService.java > % UserService > main(String[])

```
1 import com.sun.net.httpserver.*;
2 import java.io.*;
3 import java.net.InetSocketAddress;
4 import java.net.URLDecoder;
5 import java.nio.charset.StandardCharsets;
6 import java.nio.file.*;
7 import java.time.ZonedDateTime;
8 import java.util.*;
9 import java.util.regex.*;
10
11 public class UserService {
12     private static final int PORT = 9001;
13     private static final Path DATA_FILE = Paths.get(first: "data", ...more: "users.json");
14     private static final Map<String, Map<String, Object>> DB = Collections.synchronizedMap(new LinkedHashMap<>());
15
16     Run | Debug
17     public static void main(String[] args) throws Exception {
18         ensureDataFile();
19         loadFromDisk();
20
21         HttpServer server = HttpServer.create(new InetSocketAddress(PORT), backlog: 0);
22
23         server.createContext(path: "/usuarios", exchange -> {
24             addCORS(exchange);
25             try {
26                 String method = exchange.getRequestMethod();
27                 String path = exchange.getRequestURI().getPath(); // /usuarios o /usuarios/{id}
28                 if ("OPTIONS".equals(method)) { options(exchange); return; }
29                 if ("GET".equals(method)) {
30                     if ("/usuarios".equals(path)) {
31                         sendJSON(exchange, code: 200, toJson(DB));
32                     } else if (path.startsWith(prefix: "/usuarios/")) {
33                         String id = path.substring("/usuarios/".length()).trim();
34                         Map<String, Object> u = DB.get(id);
35                         if (u != null) sendJSON(exchange, code: 200, toJson(u));
36                         else sendJSON(exchange, code: 404, json: "{\"error\": \"Usuario no encontrado\"}");
37                     } else {
38                         sendJSON(exchange, code: 404, json: "{\"error\": \"Not found\"}");
39                     }
40                 } else {
41                     sendJSON(exchange, code: 405, json: "{\"error\": \"Method not allowed\"}");
42                 }
43             } catch (Exception e) {
44                 e.printStackTrace();
45                 sendJSON(exchange, code: 500, json: "{\"error\": \"Internal error\"}");
46             } finally { exchange.close(); }
47         });
48
49         System.out.println("UserService en http://127.0.0.1:" + PORT);
50         server.start();
51     }
52
53     // ===== Utils =====
54     private static void ensureDataFile() throws IOException {
55         if (!Files.exists(DATA_FILE.getParent())) Files.createDirectories(DATA_FILE.getParent());
56         if (!Files.exists(DATA_FILE)) {
```

```

56         if (!Files.exists(DATA_FILE)) {
57             String seed = "{\n" +
58                 "  \"1\": {\"id\": \"1\", \"nombre\": \"Michelle\", \"ciudad\": \"CDMX\"},\n" +
59                 "  \"2\": {\"id\": \"2\", \"nombre\": \"Rubén\", \"ciudad\": \"Guadalajara\"},\n" +
60                 "  \"3\": {\"id\": \"3\", \"nombre\": \"Sunem\", \"ciudad\": \"Monterrey\"}\n" +
61                 "}";
62             Files.writeString(DATA_FILE, seed, StandardCharsets.UTF_8);
63         }
64     }
65
66     private static void loadFromDisk() throws IOException {
67         String raw = Files.readString(DATA_FILE, StandardCharsets.UTF_8);
68         // Parseo plano de objetos {"k":{...}}
69         Pattern entry = Pattern.compile(regex: "\"(.*)\"\\s*:\\s*\\{(.*)\\}", Pattern.DOTALL);
70         Matcher m = entry.matcher(raw);
71         DB.clear();
72         while (m.find()) {
73             String id = m.group(group: 1);
74             String obj = "{" + m.group(group: 2) + "}";
75             Map<String, Object> map = parseJsonFlat(obj);
76             DB.put(id, map);
77         }
78     }
79
80     private static Map<String, Object> parseJsonFlat(String json) {
81         Map<String, Object> map = new LinkedHashMap<>();
82         // "key": "value"
83         Matcher ms = Pattern.compile(regex: "\"(.*)\"\\s*:\\s*\"(.*)\"", Pattern.DOTALL).matcher(json);
84         while (ms.find()) map.put(ms.group(group: 1), unesc(ms.group(group: 2)));
85         // "key": number/bool
86         Matcher mn = Pattern.compile(regex: "\"(.*)\"\\s*:\\s*(\\d+|true|false)").matcher(json);
87         while (mn.find()) map.put(mn.group(group: 1), mn.group(group: 2));
88         return map;
89     }
90
91     private static String toJson(Map<String, ?> m) {
92         StringBuilder sb = new StringBuilder(str: "{");
93         boolean first = true;
94         for (var e : m.entrySet()) {
95             if (!first) sb.append(str: ",");
96             first = false;
97             sb.append(str: "\"" + e.getKey() + "\"").append(str: ":");
98             sb.append(objToJson(e.getValue()));
99         }
100         sb.append(str: "}");
101         return sb.toString();
102     }
103
104     private static String objToJson(Object v) {
105         if (v instanceof Map<?, ?> mm) {
106             StringBuilder sb = new StringBuilder(str: "{");
107             boolean first = true;
108             for (var e : mm.entrySet()) {
109                 if (!first) sb.append(str: ",");
110                 first = false;

```

```

110         sb.append(str: "\"").append(esc(String.valueOf(e.getKey()))).append(str: "\":");
111         sb.append(objToJson(e.getValue()));
112     }
113     sb.append(str: "}");
114     return sb.toString();
115 } else {
116     String s = String.valueOf(v);
117     if (s.equals(anObject: "true") || s.equals(anObject: "false") || s.matches(regex: "\\d+")) return s;
118     return "\"" + esc(s) + "\"";
119 }
120 }
121
122 private static String esc(String s){ return s.replace(target: "\"",replacement: "\\\"").replace(target: "\\",replacement: "\\");
123 private static String unesc(String s){ return s.replace(target: "\\\"",replacement: "\"").replace(target: "\\\"",replacement: "\\");
124
125 private static void addCORS(HttpExchange ex) {
126     Headers h = ex.getResponseHeaders();
127     h.add(key: "Access-Control-Allow-Origin", value: "*");
128     h.add(key: "Access-Control-Allow-Headers", value: "Content-Type");
129     h.add(key: "Access-Control-Allow-Methods", value: "GET,OPTIONS");
130     h.add(key: "Cache-Control", value: "no-store");
131     h.add(key: "Date", ZonedDateTime.now().toString());
132 }
133 private static void options(HttpExchange ex) throws IOException { ex.sendResponseHeaders(rCode: 204, -1); }
134 private static void sendJSON(HttpExchange ex, int code, String json) throws IOException {
135     byte[] bytes = json.getBytes(StandardCharsets.UTF_8);
136     ex.getResponseHeaders().add(key: "Content-Type",value: "application/json; charset=utf-8");
137     ex.sendResponseHeaders(code, bytes.length);
138     try (OutputStream os = ex.getResponseBody()) { os.write(bytes); }
139 }
140 }

```

weather-service > data >  weather.json > ...

```

1 {
2   "CDMX": { "ciudad": "CDMX", "temperatura": "22°C", "condicion": "Soleado" },
3   "Guadalajara": { "ciudad": "Guadalajara", "temperatura": "24°C", "condicion": "Parcialmente nublado" },
4   "Monterrey": { "ciudad": "Monterrey", "temperatura": "26°C", "condicion": "Caluroso" }
5 }

```

weather-service > src >  WeatherService.java > ...

```

1 import com.sun.net.httpserver.*;
2 import java.io.*;
3 import java.net.InetSocketAddress;
4 import java.nio.charset.StandardCharsets;
5 import java.nio.file.*;
6 import java.time.ZonedDateTime;
7 import java.util.*;
8 import java.util.regex.*;
9
10 public class WeatherService {
11     private static final int PORT = 9002;
12     private static final Path DATA_FILE = Paths.get(first: "data", ...more: "weather.json");
13     private static final Map<String, Map<String, Object>> DB = Collections.synchronizedMap(new LinkedHashMap<>());
14
15     Run | Debug
16     public static void main(String[] args) throws Exception {
17         ensureDataFile();
18         loadFromDisk();
19
20         HttpServer server = HttpServer.create(new InetSocketAddress(PORT), backlog: 0);
21
22         server.createContext(path: "/clima", exchange -> {
23             addCORS(exchange);
24             try {
25                 String method = exchange.getRequestMethod();
26                 if ("OPTIONS".equals(method)) { options(exchange); return; }
27
28                 if (!"GET".equals(method)) { sendJSON(exchange, code: 405, json: "{\"error\":\"Method not allowed\"}");

```

```

29     String path = exchange.getRequestURI().getPath(); // /clima o /clima/CDMX
30     if ("/clima".equals(path)) {
31         sendJSON(exchange, code: 200, toJson(DB));
32     } else if (path.startsWith(prefix: "/clima/")) {
33         String ciudad = path.substring("/clima/".length()).trim();
34         Map<String, Object> w = DB.get(ciudad);
35         if (w != null) sendJSON(exchange, code: 200, toJson(w));
36         else sendJSON(exchange, code: 404, json: "{\"error\": \"Clima no disponible\"}");
37     } else {
38         sendJSON(exchange, code: 404, json: "{\"error\": \"Not found\"}");
39     }
40 } catch (Exception e) {
41     e.printStackTrace();
42     sendJSON(exchange, code: 500, json: "{\"error\": \"Internal error\"}");
43 } finally { exchange.close(); }
44 });
45
46 System.out.println("WeatherService en http://127.0.0.1:" + PORT);
47 server.start();
48 }
49
50 // ===== Utils =====
51 private static void ensureDataFile() throws IOException {
52     if (!Files.exists(DATA_FILE.getParent())) Files.createDirectories(DATA_FILE.getParent());
53     if (!Files.exists(DATA_FILE)) {
54         String seed = "{\n" +
55             "  \"CDMX\": {\n\"ciudad\": \"CDMX\", \"temperatura\": \"22°C\", \"condicion\": \"Soleado\"},\n" +
56             "  \"Guadalajara\": {\n\"ciudad\": \"Guadalajara\", \"temperatura\": \"24°C\", \"condicion\": \"Parcialm\n" +
57             "  \"Monterrey\": {\n\"ciudad\": \"Monterrey\", \"temperatura\": \"26°C\", \"condicion\": \"Caluroso\"}\n" +
58             "};\n";
59         Files.writeString(DATA_FILE, seed, StandardCharsets.UTF_8);
60     }
61 }
62
63 private static void loadFromDisk() throws IOException {
64     String raw = Files.readString(DATA_FILE, StandardCharsets.UTF_8);
65     Pattern entry = Pattern.compile(regex: "\\\"(.*)\\\"\\s*:\\s*\\\"{(.*)}\\\"\"", Pattern.DOTALL);
66     Matcher m = entry.matcher(raw);
67     DB.clear();
68     while (m.find()) {
69         String city = m.group(group: 1);
70         String obj = "{" + m.group(group: 2) + "}";
71         Map<String, Object> map = parseJsonFlat(obj);
72         DB.put(city, map);
73     }
74 }
75
76 private static Map<String, Object> parseJsonFlat(String json) {
77     Map<String, Object> map = new LinkedHashMap<>();
78     Matcher ms = Pattern.compile(regex: "\\\"(.*)\\\"\\s*:\\s*\\\"(.*)\\\"\"", Pattern.DOTALL).matcher(json);
79     while (ms.find()) map.put(ms.group(group: 1), unesc(ms.group(group: 2)));
80     Matcher mn = Pattern.compile(regex: "\\\"(.*)\\\"\\s*:\\s*(\\d+|true|false)").matcher(json);
81     while (mn.find()) map.put(mn.group(group: 1), mn.group(group: 2));

```



```

85     private static String toJson(Map<String,?> m) {
86         StringBuilder sb = new StringBuilder(str: "{");
87         boolean first = true;
88         for (var e : m.entrySet()) {
89             if (!first) sb.append(str: ","");
90             first = false;
91             sb.append(str: "\"").append(esc(e.getKey())).append(str: "\":");
92             sb.append(objToJson(e.getValue()));
93         }
94         sb.append(str: "}");
95         return sb.toString();
96     }
97     private static String objToJson(Object v) {
98         if (v instanceof Map<?,?> mm) {
99             StringBuilder sb = new StringBuilder(str: "{");
100             boolean first = true;
101             for (var e : mm.entrySet()) {
102                 if (!first) sb.append(str: ","");
103                 first = false;
104                 sb.append(str: "\"").append(esc(String.valueOf(e.getKey()))).append(str: "\":");
105                 sb.append(objToJson(e.getValue()));
106             }
107             sb.append(str: "}");
108             return sb.toString();
109         } else {
110             String s = String.valueOf(v);
111             if (s.equals(anObject: "true") || s.equals(anObject: "false") || s.matches(regex: "\\d+")) return s;
112             else {
113                 String s = String.valueOf(v);
114                 if (s.equals(anObject: "true") || s.equals(anObject: "false") || s.matches(regex: "\\d+")) return s;
115                 return "\"" + esc(s) + "\"";
116             }
117         }
118     }
119     private static String esc(String s){ return s.replace(target: "\"",replacement: "\\\"").replace(target: "\\",replacement: "\\");
120     private static String unesc(String s){ return s.replace(target: "\\\"",replacement: "\"").replace(target: "\\\\",replacement: "\\");
121     private static void addCORS(HttpExchange ex) {
122         Headers h = ex.getResponseHeaders();
123         h.add(key: "Access-Control-Allow-Origin", value: "*");
124         h.add(key: "Access-Control-Allow-Headers", value: "Content-Type");
125         h.add(key: "Access-Control-Allow-Methods", value: "GET,OPTIONS");
126         h.add(key: "Cache-Control", value: "no-store");
127         h.add(key: "Date", ZonedDateTime.now().toString());
128     }
129     private static void options(HttpExchange ex) throws IOException { ex.sendResponseHeaders(rCode: 204, -1); }
130     private static void sendJSON(HttpExchange ex, int code, String json) throws IOException {
131         byte[] bytes = json.getBytes(StandardCharsets.UTF_8);
132         ex.getResponseHeaders().add(key: "Content-Type",value: "application/json; charset=utf-8");
133         ex.sendResponseHeaders(code, bytes.length);
134         try (OutputStream os = ex.getResponseBody()) { os.write(bytes); }
135     }

```